

Xeno AI Campaign Intelligence Platform – Final Project Walkthrough

Product Introduction

Hello everyone, I'm Akshat Srivastava, a 4th year student of Bachelor of Technology in ABES Engineering College, Ghaziabad. I made this project of AI- Native CRM for my Xeno Assignment.

While exploring CRM and marketing platforms, I noticed that marketers spend a significant amount of time manually analyzing customer data, creating audience segments, planning campaigns, writing marketing content, and monitoring performance. Most existing platforms provide data dashboards, but marketers still have to make all strategic decisions themselves.

To solve this problem, I built **Xeno AI – an AI-Native Campaign Intelligence Platform for D2C brands**.

The goal was simple:

Instead of marketers deciding who to target, what campaign to run, and what message to send, AI agents should assist throughout the entire marketing lifecycle.

The platform combines CRM, customer segmentation, campaign automation, analytics, and autonomous AI agents into a single intelligent system capable of making data-driven marketing recommendations.

Functional Demo

Let me walk you through the product from a user's perspective.

Dashboard

The user first lands on the Dashboard, which provides a high-level overview of business performance through metrics such as:

- Total Customers
- Active Campaigns
- Messages Sent
- Revenue Generated

It also provides quick actions for customer import, segment creation, campaign launching, and AI-powered insights.

Customer Management

The Customers module acts as a lightweight CRM.

Users can:

- Import customers through CSV or Excel
- Search and filter customer records
- View purchase history

- Analyze customer lifetime value
- Track customer engagement

This customer data becomes the foundation for all AI-driven workflows.

Audience Segmentation

One of the most important challenges in marketing is identifying the right audience.

Users can either create rule-based segments manually using conditions such as:

- LTV > ₹10,000
- Last purchase > 60 days ago
- City = Mumbai

Or they can ask AI to automatically generate meaningful segments.

The AI can identify:

- VIP Customers
- Repeat Buyers
- New Customers
- At-Risk Customers
- High-Value Dormant Customers

This eliminates the need for manual customer analysis.

AI Campaign Studio

This is the flagship feature of the platform.

A marketer can simply type:

"Create a WhatsApp campaign for VIP customers who have not purchased in 60 days."

The AI then:

1. Understands the intent
2. Identifies the audience
3. Creates campaign strategy
4. Generates marketing copy
5. Produces multiple message variants
6. Recommends the best communication channel

The entire process happens through a conversational interface.

Opportunity Discovery

The platform also contains an AI-powered Opportunity Scanner.

Instead of marketers manually searching for growth opportunities, the AI continuously analyzes:

- Customer behaviour
- Purchase patterns

- Segment distributions
- Historical campaign performance
- Current market trends

and automatically generates opportunities such as:

- Customer reactivation campaigns
- Cross-selling opportunities
- Seasonal marketing campaigns

Each recommendation includes estimated revenue impact and AI-generated reasoning.

Campaign Execution & Analytics

Once approved, campaigns can be launched through:

- WhatsApp
- SMS
- Email
- RCS

The Pipeline Monitor tracks every stage:

- Sent
- Delivered
- Opened
- Read
- Clicked
- Converted

Finally, the Analytics Dashboard provides funnel analysis, channel performance, campaign insights, and revenue attribution.

Technical Architecture

The platform follows a **microservice architecture** consisting of four independent services.

Frontend Service

Built using:

- React
- Vite
- Tailwind CSS
- ShadCN UI

The frontend is responsible for dashboards, customer management, analytics visualization, and AI interaction interfaces.

Backend Service

Built using:

- Node.js
- Express
- MongoDB

The backend acts as the central orchestrator.

It handles:

- Authentication
- CRUD operations
- Business logic
- Analytics aggregation
- Campaign orchestration

and serves as the communication layer between all services.

Agent Service

The most interesting component is the Agent Service.

Built using:

- Python
- FastAPI
- CrewAI

Rather than relying on a single AI agent, I designed multiple specialized agents:

- Campaign Strategist
- CRM Command Agent
- Segment Generator
- Opportunity Scanner
- Insights Analyst

The Agent Service uses **CrewAI's ReAct framework**.

ReAct stands for:

Reasoning + Acting

Instead of generating responses directly, agents follow a reasoning loop:

Think → Decide → Use Tool → Observe Result → Continue Reasoning → Generate Output

For example, when a user asks:

"Show me the top-performing campaigns and suggest a retention strategy."

The agent:

1. Reasons about what information is needed.
2. Calls analytics tools.
3. Observes campaign performance data.
4. Analyzes the results.
5. Generates recommendations.

This creates more reliable and data-driven outputs compared to simple prompt-based systems.

Tavily Search Integration

One challenge with LLMs is that they only know information contained in their training data.

To solve this, I integrated **Tavily Search** into the Opportunity Scanner Agent.

When performing opportunity discovery, the agent:

- Analyzes CRM data
- Searches real-world marketing trends
- Retrieves external insights
- Combines internal and external knowledge

This allows recommendations to be informed by both customer behaviour and current market trends.

For example, if a new engagement strategy becomes popular in D2C marketing, the AI can incorporate that information into its recommendations.

Channel Service

The Channel Service simulates communication delivery through:

- WhatsApp
- SMS
- Email
- RCS

and asynchronously sends status updates back to the platform.

Database Layer

MongoDB stores:

- Customers
- Orders
- Segments
- Campaigns
- Communications
- Opportunities
- Agent Proposals
- Pipeline Events
- Settings

forming the central source of truth for the platform.

Key Engineering Decisions & Tradeoffs

While building the platform, I made several conscious engineering decisions.

Microservices vs Monolithic Architecture

I chose a microservice architecture instead of a monolith.

The primary reason was separation of concerns.

AI workloads, campaign delivery workloads, and business APIs all have very different scaling requirements.

Microservices allow:

- Independent deployment
- Independent scaling
- Better maintainability
- Easier future upgrades

For example:

- AI services can scale independently.
- Delivery services can scale during campaign spikes.
- Backend APIs remain unaffected.

The tradeoff is increased deployment and communication complexity.

For an MVP, a monolith would have been simpler, but I intentionally chose microservices to demonstrate a production-oriented architecture.

MongoDB vs PostgreSQL

I chose MongoDB over PostgreSQL.

The platform deals heavily with:

- AI-generated responses
- Agent outputs
- Dynamic customer attributes
- Flexible campaign metadata

MongoDB's document model allowed me to store evolving and nested structures naturally without frequent schema migrations.

The tradeoff is that PostgreSQL provides stronger relational consistency and transactional guarantees.

For an AI-native CRM where schemas evolve rapidly, MongoDB offered faster development and greater flexibility.

Why No Message Queue?

Currently, services communicate using HTTP APIs.

I deliberately did not introduce Kafka or RabbitMQ because the project scale did not justify the additional operational complexity.

However, at enterprise scale, I would introduce:

- Kafka
- RabbitMQ
- AWS SQS

to support:

- Event-driven communication

- Retry mechanisms
- Traffic buffering
- Better fault tolerance

For example:

Campaign Launch → Queue → Workers → Channel Service

instead of direct service-to-service communication.

This would significantly improve reliability under heavy load.

Why CrewAI Instead of a Single Agent?

The platform contains multiple business responsibilities:

- Campaign Generation
- Segmentation
- CRM Operations
- Opportunity Discovery
- Analytics

Using a single agent would create a very large prompt and reduce maintainability.

CrewAI allowed me to create specialized agents with dedicated responsibilities and tool access.

Benefits include:

- Better separation of concerns
- Easier debugging
- Improved scalability
- More reliable outputs

Each agent behaves like a domain expert within the system.

Free/Open Models vs Premium Frontier Models

The platform currently uses free or low-cost OpenAI-compatible APIs rather than expensive frontier models such as GPT-4o, Claude Opus, or Gemini Ultra.

The objective was to validate the architecture while keeping operational costs low.

Benefits:

- Lower infrastructure cost
- Faster experimentation
- Startup-friendly deployment

The tradeoff is reduced reasoning quality compared to premium models.

For production deployment, I would introduce a model abstraction layer that allows seamless switching between:

- Open-source models
- Self-hosted models
- Premium commercial models

without changing the application architecture.

Human-in-the-Loop Design

Another important design decision was requiring human approval before critical actions.

AI agents cannot directly:

- Launch campaigns
- Delete customer data
- Modify records

Instead, they create proposals that require approval.

This balances automation with business safety and mirrors how enterprise AI systems are deployed in production.

Code Walkthrough

The codebase is organized into four major modules.

Frontend

Contains:

- Pages
- Components
- Hooks
- Utility modules

A notable implementation is the Server-Sent Events hook, which enables real-time AI response streaming.

Backend

The backend follows a modular architecture with:

- Routes
- Models
- Middleware
- Services

Important services include:

Segmentation Service

Converts business rules into MongoDB queries.

Campaign Launcher

Resolves audiences, personalizes messages, and dispatches communications.

Analytics Engine

Aggregates customer and campaign metrics into business insights.

Agent Service

Contains:

- Crew definitions
- Agent definitions
- Tool integrations
- Response schemas

Each CrewAI agent has clearly defined responsibilities and controlled tool access.

Channel Service

Simulates message delivery and updates campaign statistics through asynchronous callbacks.

AI-Native Development Workflow

This project was not only AI-powered but also built using an AI-native development process.

Planning & Architecture

AI helped evaluate:

- Architecture choices
 - Database designs
 - Agent workflows
 - Service boundaries
-

Development

AI accelerated implementation of:

- CRUD APIs
- Database schemas
- React components
- Validation logic
- Error handling

allowing faster iteration cycles.

Agent Engineering

A significant amount of effort was spent designing agentic workflows.

Using CrewAI and the ReAct paradigm, agents can reason, call tools, observe results, and continue reasoning before generating outputs.

This is fundamentally different from simply calling an LLM API.

External Knowledge Integration

Tavily Search was integrated to provide external knowledge retrieval, allowing agents to combine CRM data with real-world market intelligence.

This transformed the platform from a chatbot into an actual decision-support system.

Testing & Documentation

AI also assisted with:

- Test generation
- Edge-case discovery
- Debugging
- Technical documentation

Rather than replacing engineering decisions, AI acted as a development partner that accelerated implementation while allowing me to focus on architecture and product design.

By combining CRM, campaign automation, analytics, CrewAI-based agents, ReAct reasoning, Tavily-powered external search, and a scalable microservice architecture, the platform helps businesses discover opportunities, generate campaigns, engage customers, and measure outcomes through an intelligent and production-oriented system.

Thank you.